



Green University of Bangladesh

*Department of Computer Science and Engineering (CSE)
Semester: (Summer, Year: 2026), B.Sc. in CSE (Day)*

NetGuard: Intelligent Network Security Monitoring System

*Course Title: Computer Networking Lab
Course Code: CSE-318
Section: 241-D1*

Students Details

Name	ID
Ibrahim Rahman	241002027
Ziham Mahmud	241002007

*Submission Date: 30/08/2026
Course Teacher's Name: Ms. Tanpia Tasnim*

[For teachers use only: **Don't write anything inside this box**]

<u>Lab Project Status</u>	
Marks:	Signature:
Comments:	Date:

Contents

1	Introduction	1
1.1	Introduction	1
1.2	Background	1
1.3	Project Motivation	2
1.4	Problem Statement	2
1.5	Project Objectives	3
1.6	Project Scope	4
1.7	Project Significance	4
2	System Requirements	5
2.1	Functional Requirements	5
2.2	Non-Functional Requirements	6
2.3	Hardware Requirements	6
2.4	Software Requirements	7
2.5	Development Environment	7
3	System Architecture	8
3.1	Architecture Overview	8
3.2	Major Components	9
3.3	Server Architecture	9
3.4	Client Architecture	9
3.5	Monitoring Architecture	10
3.6	Logging Architecture	10
4	Project Structure	11
4.1	Project Directory	11
4.2	ServerMain.java	12
4.3	Server.java	12

4.4	ClientHandler.java	12
4.5	SecurityMonitor.java	12
4.6	HeartbeatManager.java	13
4.7	HeartbeatSender.java	13
4.8	LogManager.java	13
4.9	ClientInfo.java	13
5	TCP Socket Communication	14
5.1	Introduction	14
5.2	ServerSocket	14
5.3	Socket	14
5.4	DataInputStream	15
5.5	DataOutputStream	15
5.6	Communication Sequence	15
5.7	Local Testing	15
6	Client-Server Implementation	16
6.1	Server Startup	16
6.2	Client Acceptance	17
6.3	Client Identification	18
6.4	Client Information Creation	18
6.5	Client List	18
6.6	Multiple Clients	19
6.7	Connection Lifecycle	19
7	Heartbeat Monitoring	20
7.1	Purpose of Heartbeat	20
7.2	Heartbeat Message	20
7.3	Heartbeat Sender	20
7.4	Heartbeat Reception	20
7.5	Last Heartbeat	21
7.6	HeartbeatManager	21
7.7	Timeout Detection	21
7.8	Heartbeat Benefits	22
7.9	Heartbeat Limitation	22

8	Security Monitoring	23
8.1	Security Monitoring Overview	23
8.2	Connection Attempt Monitoring	23
8.3	Warning Threshold	23
8.4	Connection Alert	24
8.5	Disconnect Monitoring	24
8.6	Frequent Disconnect Alert	24
8.7	Security Alert Flow	24
8.8	Security Monitor Example	25
8.9	Disconnect Alert Example	25
9	Graphical User Interface	26
9.1	GUI Overview	26
9.2	Server Status	26
9.3	Client Information	26
9.4	Live Activity	27
9.5	Security Alert Display	27
9.6	Thread Safety	27
9.7	GUI Advantages	28
10	Activity Logging	29
10.1	Introduction	29
10.2	Log Directory	29
10.3	Activity Log File	29
10.4	LogManager Initialization	29
10.5	Logging Method	30
10.6	Timestamp	30
10.7	Security Alert Logging	30
10.8	Logging Test	30
10.9	Importance of Activity Log	31
11	Multithreading	32
11.1	Introduction	32
11.2	Server Thread	32
11.3	Client Handler Threads	33

11.4 Heartbeat Thread	33
11.5 Client Heartbeat Thread	33
11.6 Benefits	33
11.7 Synchronization	34
12 Algorithms and Workflows	35
12.1 Server Startup Algorithm	35
12.2 Client Connection Algorithm	36
12.3 Heartbeat Algorithm	36
12.4 Disconnection Algorithm	36
12.5 Security Alert Algorithm	37
12.6 Activity Logging Algorithm	37
13 Testing and Results	38
13.1 Testing Introduction	38
13.2 Test 1: Server Startup	38
13.3 Test 2: Client Connection	39
13.4 Test 3: Heartbeat	40
13.5 Test 4: Client Disconnect	41
13.6 Test 5: Security Alert	41
13.7 Activity Log Testing	42
13.8 Overall Test Summary	42
13.9 Testing Conclusion	42
14 Security Analysis	43
14.1 Security Features	43
14.2 Repeated Connection Detection	43
14.3 Frequent Disconnect Detection	43
14.4 Advantages	44
14.5 Limitations of Security Mechanism	44
15 Performance and Reliability	45
15.1 Multithreaded Performance	45
15.2 Heartbeat Performance	45
15.3 Timeout Performance	45
15.4 GUI Responsiveness	45

15.5 Resource Handling	46
15.6 Reliability	46
15.7 Current Scale	46
16 Limitations	47
16.1 No Authentication	47
16.2 No Encryption	47
16.3 Basic Security Detection	47
16.4 Limited Historical Analysis	47
16.5 No Automatic Blocking	47
16.6 Local Testing Focus	48
16.7 Academic Scope	48
17 Future Improvements	49
17.1 Authentication	49
17.2 Encryption	49
17.3 Database Integration	49
17.4 Advanced Security Detection	50
17.5 Automatic Blocking	50
17.6 Graphical Statistics	50
17.7 Remote Monitoring	50
17.8 Notification System	50
18 Output pictures	51
18.1 Full GUI	51
18.2 Client Interface	52
18.3 Total client, online client, security alert, heartbeat received	52
18.4 Live activities log, server control, and security alert	52
19 Discussion	53
20 Conclusion	54
A Important Source Code	55
A.1 Server Port	55
A.2 Server Socket	55
A.3 Client Acceptance	55

A.4	Heartbeat Sender	56
A.5	Heartbeat Timeout	57
A.6	Security Threshold	57
A.7	Security Alert	58
A.8	Offline Handling	58
B	LogManager Source Code	59
C	Final Test Evidence	63
C.1	Server Startup Evidence	63
C.2	Client Connection Evidence	63
C.3	Heartbeat Evidence	64
C.4	Offline Evidence	64
C.5	Security Alert Evidence	64
C.6	Logging Evidence	65
D	Final Demonstration Checklist	66
E	CEA and CEP	69

Chapter 1

Introduction

1.1 Introduction

Computer networks are an important part of modern computing systems. Multiple computers, mobile devices, servers, and other devices can communicate with each other through networks. As the number of connected devices increases, monitoring these devices becomes more important.

A network administrator needs to know which clients are connected, which clients are active, when a client becomes unavailable, and whether unusual connection behavior is occurring.

A simple client-server application can establish communication between two devices, but it may not provide sufficient monitoring information. NetGuard was developed to extend the basic concept of client-server communication into a small network monitoring and security monitoring system.

The system uses Java TCP sockets for communication. A central server accepts clients and creates separate handler threads for them. Each client periodically sends heartbeat messages so that the server can determine whether the client is still active.

1.2 Background

Networking is a fundamental area of computer science. TCP/IP-based communication is used by many applications and services. Socket programming provides a programming interface through which applications can communicate across a network.

In a typical client-server model, the server waits for connections while clients initiate communication. The server can then process multiple clients concurrently.

However, simply accepting connections does not provide continuous monitoring. A client may suddenly close, lose its connection, or stop responding. This is why a heartbeat mechanism can be useful.

NetGuard combines these concepts into a single application.

1.3 Project Motivation

The motivation behind NetGuard is to develop a practical project that demonstrates several networking concepts together.

The major motivations are:

- To understand TCP socket programming.
- To implement client-server communication.
- To handle multiple clients using threads.
- To monitor client connection status.
- To implement a heartbeat mechanism.
- To detect inactive clients.
- To detect repeated connection attempts.
- To detect frequent connection and disconnection behavior.
- To provide security alerts.
- To display monitoring information through a GUI.
- To store important activities in a log file.

1.4 Problem Statement

A basic network server normally provides connection functionality. However, connection alone does not provide complete information about the health of clients.

For example, a client may connect successfully and later disconnect. If the server does not monitor the client continuously, it may not immediately recognize the change.

Another problem is repeated connection behavior. A client repeatedly connecting and disconnecting may be normal during testing, but such behavior can also indicate suspicious activity.

NetGuard addresses these problems by introducing client monitoring, heartbeat messages, connection counters, disconnect counters, security alerts, GUI monitoring, and persistent logging.

1.5 Project Objectives

The main objectives of NetGuard are:

1. To create a Java-based TCP client-server application.
2. To allow the server to accept multiple clients.
3. To identify connected clients.
4. To store client IP addresses.
5. To record connection time.
6. To monitor client connection status.
7. To implement heartbeat communication.
8. To detect heartbeat timeout.
9. To detect client disconnection.
10. To track repeated connection attempts.
11. To track disconnect behavior.
12. To generate security alerts.
13. To display security alerts through GUI.
14. To maintain live activity information.
15. To store important events in an activity log file.

1.6 Project Scope

The project focuses on a small-scale network monitoring environment. It is designed mainly as an academic Computer Networking project.

The project scope includes:

- TCP communication.
- Client-server architecture.
- Java socket programming.
- Multithreaded client handling.
- Heartbeat monitoring.
- Client status management.
- Basic security behavior detection.
- Security alert generation.
- GUI-based monitoring.
- Activity logging.

The system is not intended to replace enterprise-level security products.

1.7 Project Significance

The project demonstrates how different networking concepts can be integrated into one working application.

It is particularly useful for understanding the relationship between network communication and network monitoring.

The project also demonstrates that security monitoring does not always require complex packet analysis. Simple behavioral indicators, such as repeated connections, can be used to generate alerts.

Chapter 2

System Requirements

2.1 Functional Requirements

The functional requirements describe what the system should do.

1. The server shall start successfully.
2. The server shall listen on TCP port 5000.
3. The server shall accept incoming client connections.
4. The server shall receive the client name.
5. The server shall identify the client IP address.
6. The server shall record connection time.
7. The server shall maintain client information.
8. The client shall send heartbeat messages.
9. The server shall receive heartbeat messages.
10. The server shall update the last heartbeat time.
11. The system shall detect client disconnection.
12. The system shall mark disconnected clients as OFFLINE.
13. The system shall track connection attempts.
14. The system shall track disconnect counts.
15. The system shall generate security alerts.
16. The GUI shall display security alerts.
17. The system shall maintain live activities.
18. The system shall save important events into a log file.

2.2 Non-Functional Requirements

Important non-functional requirements include:

- The application should be easy to use.
- The GUI should remain responsive.
- Multiple clients should be handled independently.
- Network failures should not crash the complete system.
- Important activities should be recorded.
- The source code should remain modular.
- The system should be understandable for academic demonstration.

2.3 Hardware Requirements

NetGuard does not require specialized networking hardware.

A normal personal computer can be used for development and testing.

Recommended hardware includes:

- Modern Intel or AMD processor.
- Minimum 4 GB RAM.
- At least 1 GB available storage.
- Keyboard and mouse.
- Network adapter.

For local testing, the server and client can run on the same computer using the loopback address.

2.4 Software Requirements

Table 2.1: Software Requirements

Component	Requirement
Programming Language	Java
JDK	Java 21
IDE	IntelliJ IDEA
GUI	Java Swing
Communication	Java TCP Socket API
Operating System	Windows
Build Tool	Maven where applicable

2.5 Development Environment

The project was developed using IntelliJ IDEA with Java Development Kit 21.

The Java environment provides classes such as:

- ServerSocket
- Socket
- DataInputStream
- DataOutputStream
- Thread
- SwingUtilities
- LocalDateTime
- DateTimeFormatter

Chapter 3

System Architecture

3.1 Architecture Overview

NetGuard follows a client-server architecture.

The server acts as the central monitoring point. Clients establish TCP connections with the server and communicate through socket streams.

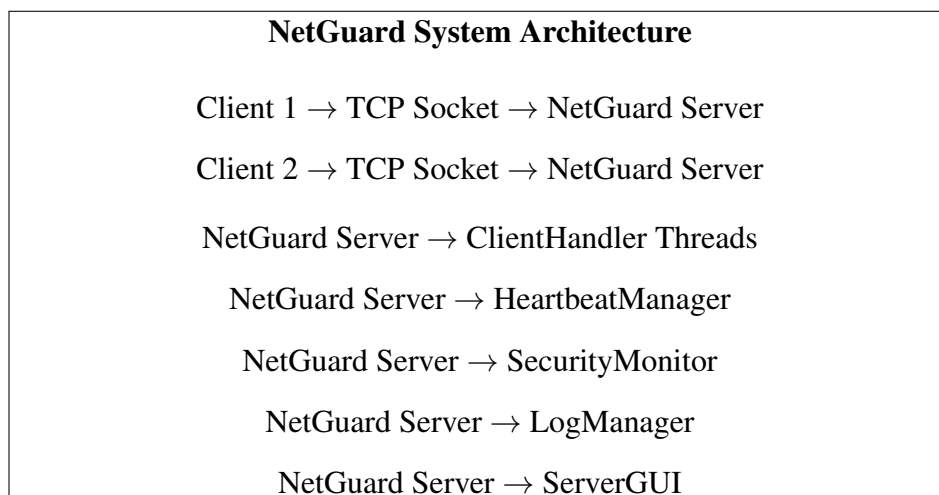


Figure 3.1: High-level architecture of NetGuard

3.2 Major Components

The major components of the system are:

- ServerMain
- Server
- ClientHandler
- Client
- HeartbeatSender
- HeartbeatManager
- SecurityMonitor
- ServerGUI
- ClientGUI
- ClientInfo
- LogManager
- Constants
- Logger

3.3 Server Architecture

The Server class creates a ServerSocket on port 5000.

The server waits for clients using the accept operation.

When a client connects, a new ClientHandler object is created.

A separate thread is started for the client.

This architecture prevents one client from blocking other clients.

3.4 Client Architecture

The client establishes a TCP connection with the server.

After connecting, it sends the client name.

The client then starts its heartbeat mechanism.

The heartbeat sender sends the following message:

HEARTBEAT

at regular intervals.

3.5 Monitoring Architecture

The monitoring layer contains two important mechanisms.

The first is HeartbeatManager, which checks client activity.

The second is SecurityMonitor, which checks behavioral patterns.

3.6 Logging Architecture

LogManager provides persistent logging.

The log directory is:

`resources/logs`

The log file is:

`resources/logs/activity.log`

Chapter 4

Project Structure

4.1 Project Directory

The project was organized into several packages.

```
NetGuard
|
+-- server
|   |
|   +-- ServerMain.java
|   +-- Server.java
|   +-- ClientHandler.java
|   +-- SecurityMonitor.java
|   +-- HeartbeatManager.java
|   +-- LogManager.java
|   +-- ServerGUI.java
|
+-- client
|   |
|   +-- ClientMain.java
|   +-- Client.java
|   +-- HeartbeatSender.java
|   +-- ClientGUI.java
|
+-- model
|   |
|   +-- ClientInfo.java
|
+-- utils
|   |
|   +-- Constants.java
|   +-- Logger.java
```

4.2 ServerMain.java

ServerMain is the main entry point for the GUI-enabled server.

The GUI is created first, and then the server is started in a separate thread.

This prevents the server's infinite accept loop from blocking the Swing GUI.

4.3 Server.java

The Server class is responsible for:

- Creating the ServerSocket.
- Listening on port 5000.
- Accepting clients.
- Creating ClientHandler objects.
- Maintaining the client list.
- Starting HeartbeatManager.
- Communicating with ServerGUI.

4.4 ClientHandler.java

ClientHandler implements Runnable.

Each client gets its own handler thread.

The handler receives the client name and heartbeat messages.

When the client disconnects, the handler changes the client status to OFFLINE.

4.5 SecurityMonitor.java

SecurityMonitor is responsible for basic security behavior detection.

It maintains:

- Connection attempt counts.
- Disconnect counts.

The warning limit is:

$$W = 3$$

When the counter reaches three, a security alert is generated.

4.6 HeartbeatManager.java

HeartbeatManager implements Runnable.

It periodically checks the last heartbeat timestamp of clients.

The implemented checking interval is five seconds and the timeout threshold is ten seconds.

4.7 HeartbeatSender.java

HeartbeatSender runs on the client side.

It sends:

HEARTBEAT

every five seconds.

4.8 LogManager.java

LogManager handles persistent logging.

It creates the log directory if necessary and appends activities to the log file.

4.9 ClientInfo.java

ClientInfo stores the information associated with a client.

Important fields include:

- Client name.
- IP address.
- Connection time.
- Connection status.
- Last heartbeat.

Chapter 5

TCP Socket Communication

5.1 Introduction

TCP socket programming is the foundation of NetGuard.

TCP provides connection-oriented communication between the client and server.

5.2 ServerSocket

The server uses:

```
1 ServerSocket serverSocket =  
2     new ServerSocket(5000);
```

The server then waits for clients.

5.3 Socket

A client establishes a socket connection with the server.

The server receives the socket through:

```
1 Socket clientSocket =  
2     serverSocket.accept();
```

5.4 DataInputStream

NetGuard uses DataInputStream to receive UTF messages.

For example:

```
1 DataInputStream input =  
2     new DataInputStream(  
3         clientSocket.getInputStream()  
4     );
```

5.5 DataOutputStream

The client uses DataOutputStream to send heartbeat messages.

```
1 output.writeUTF("HEARTBEAT");  
2 output.flush();
```

5.6 Communication Sequence

The communication sequence is:

1. Server starts.
2. Server listens on port 5000.
3. Client starts.
4. Client connects to server.
5. Client sends its name.
6. Server receives the name.
7. Server creates ClientInfo.
8. Client starts heartbeat.
9. Server receives heartbeat.
10. Server updates client information.

5.7 Local Testing

During final testing, the local IP address used was:

127.0.0.1

This allowed both client and server to run on the same computer.

Chapter 6

Client-Server Implementation

6.1 Server Startup

When ServerMain is executed, the GUI is created.

Then a separate thread starts the server.

The server displays:

```
=====
      NetGuard Server
=====
Server started successfully.
Port: 5000
Waiting for clients...
```

6.2 Client Acceptance

The server continuously executes the accept operation.

```
1 while (true) {  
2  
3     Socket clientSocket =  
4         serverSocket.accept();  
5  
6     ClientHandler clientHandler =  
7         new ClientHandler(  
8             clientSocket,  
9             this,  
10            gui  
11        );  
12  
13     Thread clientThread =  
14         new Thread(clientHandler);  
15  
16     clientThread.start();  
17 }
```

6.3 Client Identification

The client sends its name using UTF format.

The server reads:

```
1 String clientName =  
2     input.readUTF();
```

The IP address is obtained using:

```
1 String clientIP =  
2     clientSocket  
3         .getInetAddress()  
4         .getHostAddress();
```

6.4 Client Information Creation

The server creates a ClientInfo object.

The initial status is:

ONLINE

The last heartbeat initially remains:

Not received yet

6.5 Client List

The server maintains a list of ClientInfo objects.

The list allows the system to display multiple client records.

For example:

===== Connected Clients =====

```
1. Ibrahim  
   IP: 127.0.0.1  
   Connection Time: 2026-08-27 14:53:56  
   Status: ONLINE  
   Last Heartbeat: Not received yet
```

=====

Total Connected Clients: 1

6.6 Multiple Clients

The project uses a separate ClientHandler thread for every connected client.

Therefore, several clients can communicate with the server independently.

6.7 Connection Lifecycle

The complete lifecycle is:

1. Client starts.
2. Client connects.
3. Client name is received.
4. ClientInfo is created.
5. Client is added to server list.
6. GUI is updated.
7. Heartbeats are received.
8. Client eventually disconnects.
9. Client status becomes OFFLINE.

Chapter 7

Heartbeat Monitoring

7.1 Purpose of Heartbeat

A network connection alone cannot always provide a convenient application-level indication of whether a client is active.

NetGuard therefore uses heartbeat messages.

A heartbeat is a small periodic message sent by the client to indicate that it is still running and communicating with the server.

7.2 Heartbeat Message

The message used by the implementation is:

HEARTBEAT

7.3 Heartbeat Sender

HeartbeatSender sends the message every five seconds.

```
1 output.writeUTF("HEARTBEAT");  
2 output.flush();  
3  
4 Thread.sleep(5000);
```

7.4 Heartbeat Reception

The server receives messages inside ClientHandler.

When the received message equals HEARTBEAT, the server records the current time.

7.5 Last Heartbeat

The ClientInfo object stores the last heartbeat timestamp.

For example:

```
Last Heartbeat:  
2026-08-27 14:54:01
```

7.6 HeartbeatManager

HeartbeatManager runs as a separate thread.

The checking interval is:

5 seconds

The timeout threshold is:

10 seconds

7.7 Timeout Detection

The system calculates the time difference between the current time and the last heartbeat.

If:

$\text{Elapsed Time} > 10$

seconds, the client is considered inactive.

The status is then changed to:

OFFLINE

7.8 Heartbeat Benefits

The heartbeat mechanism provides:

- Continuous liveness information.
- Last communication timestamp.
- Offline detection.
- Practical demonstration of periodic communication.
- Better client monitoring.

7.9 Heartbeat Limitation

The heartbeat mechanism is a simple application-level monitoring system. It does not perform packet-level network analysis.

Chapter 8

Security Monitoring

8.1 Security Monitoring Overview

Security monitoring is one of the major features of NetGuard.

The system observes client connection behavior and attempts to identify unusual patterns.

The project does not claim to be a complete intrusion detection system. Instead, it provides a simple behavioral security layer.

8.2 Connection Attempt Monitoring

SecurityMonitor maintains a connection attempt counter for each client name.

The counter starts from zero.

Whenever a client connects, the counter increases by one.

8.3 Warning Threshold

The implemented threshold is:

$$W = 3$$

Therefore, when the number of connection attempts reaches three, the system generates a security alert.

8.4 Connection Alert

The alert message is:

```
Multiple connection attempts detected.  
Client: Ibrahim
```

8.5 Disconnect Monitoring

SecurityMonitor also stores disconnect counts.

Whenever a client disconnects, its disconnect counter is increased.

8.6 Frequent Disconnect Alert

When the disconnect count reaches three, the system generates:

```
Frequent connect/disconnect behavior detected.  
Client: Ibrahim
```

8.7 Security Alert Flow

The security alert flow is:

1. Client connects or disconnects.
2. SecurityMonitor receives the event.
3. Counter is increased.
4. Counter is compared with threshold.
5. If threshold is reached, alert is generated.
6. LogManager records the alert.
7. ServerGUI displays the alert.

8.8 Security Monitor Example

During final testing, the following output was observed:

```
Security Monitor: Checking client Ibrahim  
Connection Attempts: 3
```

```
SECURITY ALERT  
Suspicious connection behavior detected!  
Client: Ibrahim  
Multiple connection attempts detected.
```

8.9 Disconnect Alert Example

Another tested output was:

```
Security Monitor: Client disconnected  
Client: Ibrahim  
Disconnect Count: 3
```

```
SECURITY ALERT  
Frequent connect/disconnect behavior detected!  
Client: Ibrahim
```

Chapter 9

Graphical User Interface

9.1 GUI Overview

NetGuard contains a Java Swing-based graphical user interface.

The GUI allows the user to monitor the server without depending only on console output.

9.2 Server Status

When ServerMain starts, the GUI is created automatically.

The server is started in a separate thread.

This design allows the GUI to remain responsive.

9.3 Client Information

The GUI displays client information such as:

- Client name.
- IP address.
- Connection time.
- Connection status.
- Heartbeat-related information.

9.4 Live Activity

The GUI includes a live activity area.

It can show activities such as:

```
CONNECTED  
HEARTBEAT  
OFFLINE  
SECURITY ALERT  
SYSTEM  
ERROR
```

9.5 Security Alert Display

When a security alert occurs, the GUI updates the security alert counter and displays the alert message.

The implemented method uses `SwingUtilities.invokeLater` to update GUI components safely.

9.6 Thread Safety

Networking operations are performed in background threads.

Swing components should be updated on the Swing event dispatch thread.

Therefore, the implementation uses:

```
1 SwingUtilities.invokeLater(  
2     () -> {  
3         // GUI update  
4     }  
5 );
```

9.7 GUI Advantages

The GUI provides:

- Better visualization.
- Live monitoring.
- Easy demonstration.
- Security alert visibility.
- Client status visibility.
- Activity monitoring.

Chapter 10

Activity Logging

10.1 Introduction

NetGuard includes a LogManager component for persistent activity logging.

The GUI provides live activity information, while LogManager keeps important events in a file.

10.2 Log Directory

The system uses:

```
resources/logs
```

If the directory does not exist, LogManager creates it.

10.3 Activity Log File

The file is:

```
resources/logs/activity.log
```

10.4 LogManager Initialization

During testing, the application printed:

```
LogManager initialized. Log path:  
C:\Users\Jahangir\IdeaProjects\  
NetGuard_Computer_Networking_Project\  
resources\logs\activity.log
```

10.5 Logging Method

The log method writes the activity using append mode.

```
1 FileWriter fileWriter =  
2     new FileWriter(  
3         LOG_FILE ,  
4         true  
5     );
```

Using append mode means new activities are added without removing previous activities.

10.6 Timestamp

Every log entry receives a timestamp.

The format is:

yyyy-MM-dd HH:mm:ss

10.7 Security Alert Logging

Security alerts are written using:

```
1 LogManager.logSecurityAlert(  
2     alertMessage  
3 );
```

The resulting activity contains:

SECURITY ALERT |
Frequent connect/disconnect behavior detected.
Client: Ibrahim

10.8 Logging Test

During final testing, the console displayed:

LOGGING: SECURITY ALERT |
Frequent connect/disconnect behavior detected.
Client: Ibrahim

Log saved successfully.

The activity log file was also confirmed through Windows Explorer.

10.9 Importance of Activity Log

Although the GUI already provides live activities, the persistent activity log is useful because it preserves events after the live GUI session ends.

It can therefore be used as a basic historical record.

Chapter 11

Multithreading

11.1 Introduction

Multithreading is an important part of NetGuard.

A server that handles multiple clients sequentially could become blocked when one client remains connected.

NetGuard avoids this by creating a separate thread for each client.

11.2 Server Thread

When ServerMain starts the server, it creates a server thread.

```
1 Thread serverThread =  
2     new Thread(  
3         () -> {  
4             Server server =  
5                 new Server(gui);  
6  
7             server.startServer();  
8         },  
9         "NetGuard-Server "  
10    );  
11  
12 serverThread.start();
```

11.3 Client Handler Threads

Each connected client receives a separate ClientHandler thread.

```
1 Thread clientThread =
2     new Thread(
3         clientHandler ,
4         "NetGuard-Client - "
5     );
6
7 clientThread.start();
```

11.4 Heartbeat Thread

HeartbeatManager also runs independently.

```
1 Thread heartbeatThread =
2     new Thread(
3         heartbeatManager ,
4         "NetGuard-Heartbeat-Manager "
5     );
6
7 heartbeatThread.setDaemon(true);
8 heartbeatThread.start();
```

11.5 Client Heartbeat Thread

On the client side, HeartbeatSender runs independently so that heartbeat communication does not block other client activities.

11.6 Benefits

Multithreading provides:

- Concurrent client processing.
- Independent heartbeat monitoring.
- Better GUI responsiveness.
- Better server responsiveness.
- Practical demonstration of Java concurrency.

11.7 Synchronization

Shared data is protected using synchronized methods where required.

For example:

```
1 public synchronized void addClient(  
2     ClientInfo clientInfo) {  
3  
4     clients.add(clientInfo);  
5 }
```

Chapter 12

Algorithms and Workflows

12.1 Server Startup Algorithm

1. Start ServerMain.
2. Create ServerGUI.
3. Apply system look and feel.
4. Create a server thread.
5. Create Server object.
6. Start server.
7. Create ServerSocket on port 5000.
8. Notify GUI that server is running.
9. Start HeartbeatManager.
10. Wait for clients.

12.2 Client Connection Algorithm

1. Client starts.
2. Client creates socket.
3. Client connects to server.
4. Client sends its name.
5. Server receives name.
6. Server obtains IP address.
7. Server creates ClientInfo.
8. Server adds ClientInfo.
9. GUI displays client.
10. SecurityMonitor checks connection behavior.

12.3 Heartbeat Algorithm

1. Client sends HEARTBEAT.
2. Server receives HEARTBEAT.
3. Current time is generated.
4. Last heartbeat is updated.
5. Client status becomes ONLINE.
6. HeartbeatManager periodically checks the timestamp.
7. If timeout exceeds ten seconds, client becomes OFFLINE.

12.4 Disconnection Algorithm

1. Client closes or loses the socket connection.
2. ClientHandler detects communication failure.
3. Client status is changed to OFFLINE.
4. SecurityMonitor records disconnect.
5. GUI receives offline notification.
6. Activity is displayed.
7. LogManager can store the event.

12.5 Security Alert Algorithm

1. Receive connection event.
2. Identify client.
3. Increase connection counter.
4. Compare counter with three.
5. If counter is at least three, create alert.
6. Send alert to GUI.
7. Save alert to log.

12.6 Activity Logging Algorithm

1. Receive important event.
2. Check whether log directory exists.
3. Create directory if required.
4. Open activity.log in append mode.
5. Generate timestamp.
6. Write timestamp and message.
7. Close writer.
8. Display successful logging information.

Chapter 13

Testing and Results

13.1 Testing Introduction

After implementation, the project was tested using actual server and client executions.

The final testing focused on the most important project features.

The five major tests were completed successfully.

13.2 Test 1: Server Startup

Table 13.1: Test Case 1: Server Startup

Item	Result
Test Action	Run ServerMain
Expected Result	GUI opens and server starts
Actual Result	Server started successfully
Port	5000
Status	PASS

Observed output:

```
NetGuard Server
Server started successfully.
Port: 5000
Waiting for clients...
```

13.3 Test 2: Client Connection

A client named Ibrahim was connected to the server.

Observed output:

```
New client connection received.
```

```
Security Monitor:
```

```
Checking client Ibrahim
```

```
Connection Attempts: 1
```

```
Client verified: Ibrahim
```

```
===== Connected Clients =====
```

```
1. Ibrahim
```

```
IP: 127.0.0.1
```

```
Status: ONLINE
```

Result:

PASS

13.4 Test 3: Heartbeat

After the client connected, heartbeat messages were received.

Observed output:

```
Heartbeat received from Ibrahim
```

```
Client: Ibrahim  
IP: 127.0.0.1  
Status: ONLINE  
Last Heartbeat:  
2026-08-27 14:53:57
```

Another heartbeat was received:

```
Heartbeat received from Ibrahim
```

```
Client: Ibrahim  
IP: 127.0.0.1  
Status: ONLINE  
Last Heartbeat:  
2026-08-27 14:54:01
```

Result:

PASS

13.5 Test 4: Client Disconnect

The client was intentionally closed.

The server detected the disconnection.

Observed output:

```
Client disconnected: Ibrahim
Status changed to OFFLINE
```

SecurityMonitor also recorded the disconnect.

The GUI displayed the offline event.

Result:

PASS

13.6 Test 5: Security Alert

The same client was connected and disconnected repeatedly.

The connection attempt counter increased:

```
Connection Attempts: 1
Connection Attempts: 2
Connection Attempts: 3
```

At the third attempt, the security alert was generated.

Observed output:

```
SECURITY ALERT
```

```
Suspicious connection behavior detected!
```

```
Client: Ibrahim
```

```
Multiple connection attempts detected.
```

The disconnect counter also reached three.

The system generated:

```
SECURITY ALERT
```

```
Frequent connect/disconnect behavior detected!
```

```
Client: Ibrahim
```

Result:

PASS

13.7 Activity Log Testing

The LogManager was also tested.

The console showed:

```
LogManager initialized. Log path:  
...\resources\logs\activity.log
```

```
LOGGING:
```

```
SECURITY ALERT |
```

```
Frequent connect/disconnect behavior detected.
```

```
Client: Ibrahim
```

Log saved successfully.

The log file was confirmed in Windows Explorer.

Result:

PASS

13.8 Overall Test Summary

Table 13.2: Final Testing Summary

No.	Test	Result
1	Server startup	PASS
2	Client connection	PASS
3	Heartbeat reception	PASS
4	Client disconnection	PASS
5	Security alert detection	PASS
6	Activity log	PASS

13.9 Testing Conclusion

All major implemented features were successfully tested.

The final testing confirms that the project operates as expected in the controlled test environment.

Chapter 14

Security Analysis

14.1 Security Features

NetGuard contains a basic security monitoring mechanism.

The main security features are:

- Connection attempt monitoring.
- Disconnect monitoring.
- Repeated connection detection.
- Frequent connect/disconnect detection.
- Security alert generation.
- Security alert logging.
- GUI security alert display.

14.2 Repeated Connection Detection

Repeated connection attempts can be monitored using a counter.

The implemented threshold is three.

This means the third connection attempt triggers the implemented security alert condition.

14.3 Frequent Disconnect Detection

Frequent disconnect behavior is also monitored.

When the disconnect count reaches three, an alert is generated.

14.4 Advantages

The implemented security mechanism is:

- Simple.
- Easy to understand.
- Easy to demonstrate.
- Fast.
- Integrated with the GUI.
- Integrated with logging.

14.5 Limitations of Security Mechanism

The security mechanism is not a full intrusion detection system.

It does not perform:

- Packet inspection.
- Malware detection.
- Port scanning.
- Deep packet analysis.
- Intrusion prevention.
- Automatic blocking.
- Encryption analysis.

Therefore, the security component should be considered a basic behavioral monitoring module.

Chapter 15

Performance and Reliability

15.1 Multithreaded Performance

The server uses separate threads for clients.

This means that one connected client does not prevent the server from accepting another client.

15.2 Heartbeat Performance

Heartbeat messages are lightweight.

The client sends one heartbeat approximately every five seconds.

Therefore, heartbeat communication generates relatively small network traffic.

15.3 Timeout Performance

The heartbeat manager checks clients every five seconds.

The configured timeout is ten seconds.

This provides a simple balance between monitoring frequency and processing overhead.

15.4 GUI Responsiveness

The server is started in a separate thread from the GUI.

This helps keep the GUI responsive while the server waits for incoming connections.

15.5 Resource Handling

ClientHandler closes the client socket inside the finally block.

This provides basic resource cleanup after communication ends.

15.6 Reliability

The system uses exception handling around network operations.

When a client disconnects unexpectedly, ClientHandler catches the communication exception and changes the client status to OFFLINE.

15.7 Current Scale

The current implementation is suitable for academic demonstrations and small local-network tests.

For large deployments, additional mechanisms would be required.

Chapter 16

Limitations

Although the project successfully meets its main academic objectives, there are several limitations.

16.1 No Authentication

The current client-server system does not implement a complete authentication mechanism.

16.2 No Encryption

The implemented communication uses normal TCP socket streams. Encryption such as TLS is not implemented.

16.3 Basic Security Detection

SecurityMonitor uses simple threshold-based behavior detection.

It does not perform advanced intrusion analysis.

16.4 Limited Historical Analysis

The activity log provides persistent events, but the GUI does not currently provide advanced historical charts.

16.5 No Automatic Blocking

Suspicious clients are alerted but not automatically blocked.

16.6 Local Testing Focus

Most project testing was performed in a controlled local environment using the loop-back IP address.

16.7 Academic Scope

The project was designed for educational purposes and should not be considered a replacement for professional enterprise monitoring systems.

Chapter 17

Future Improvements

17.1 Authentication

A future version can implement username-password authentication or certificate-based authentication.

17.2 Encryption

TLS-based secure communication could be added to protect data transmitted between clients and server.

17.3 Database Integration

MySQL or another database could be integrated to store:

- Client information.
- Connection history.
- Security alerts.
- Disconnect history.
- Heartbeat history.

17.4 Advanced Security Detection

Future versions can introduce:

- Time-window based connection analysis.
- IP-based monitoring.
- Connection rate analysis.
- Suspicious pattern classification.
- More advanced alert levels.

17.5 Automatic Blocking

A future version could temporarily block clients that repeatedly generate suspicious events.

17.6 Graphical Statistics

Charts could be added to show:

- Number of connections.
- Number of disconnections.
- Security alert trends.
- Online/offline statistics.
- Heartbeat health.

17.7 Remote Monitoring

A future version could provide a secure remote monitoring dashboard.

17.8 Notification System

Email, desktop, or other notification mechanisms could be added for important security alerts.

Chapter 18

Output pictures

18.1 Full GUI

ServerGUI Interface graphicx

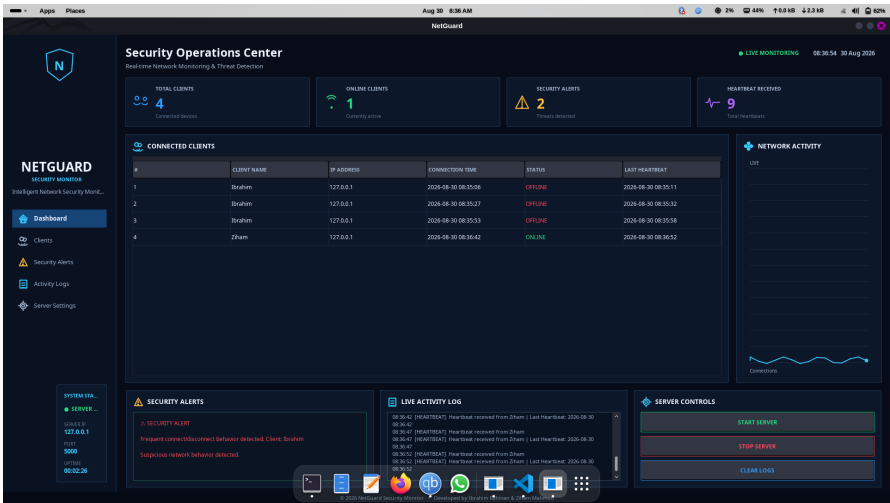
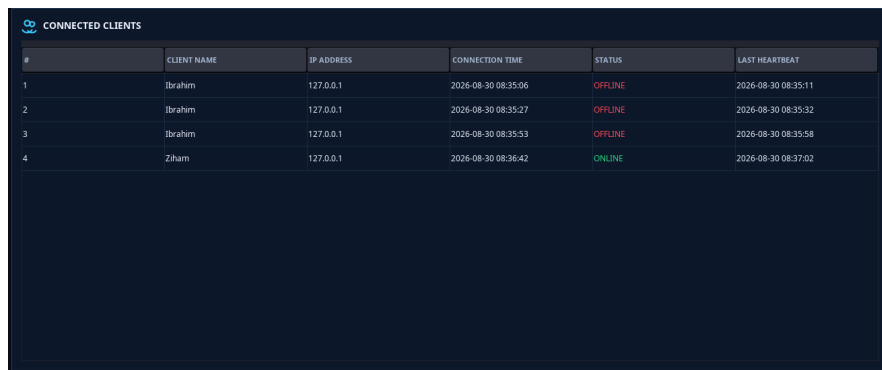


Figure 18.1: Your Image Caption

18.2 Client Interface



#	CLIENT NAME	IP ADDRESS	CONNECTION TIME	STATUS	LAST HEARTBEAT
1	Ibrahim	127.0.0.1	2026-08-30 08:35:06	OFFLINE	2026-08-30 08:35:11
2	Ibrahim	127.0.0.1	2026-08-30 08:35:27	OFFLINE	2026-08-30 08:35:32
3	Ibrahim	127.0.0.1	2026-08-30 08:35:53	OFFLINE	2026-08-30 08:35:58
4	Zham	127.0.0.1	2026-08-30 08:36:42	ONLINE	2026-08-30 08:37:02

Figure 18.2: Client List

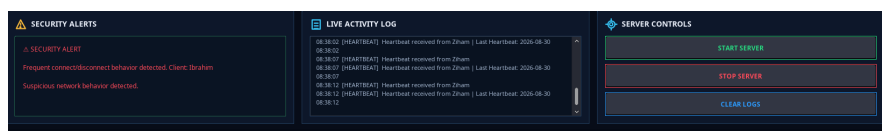
18.3 Total client, online client, security alert, heartbeat received



TOTAL CLIENTS	ONLINE CLIENTS	SECURITY ALERTS	HEARTBEAT RECEIVED
4 Connected devices	1 Currently active	2 Threats detected	18 Total Heartbeats

Figure 18.3: total client, online client, security alert, heartbeat received

18.4 Live activities log, server control, and security alert



SECURITY ALERTS	LIVE ACTIVITY LOG	SERVER CONTROLS
SECURITY ALERT Frequent connect/disconnect behavior detected. Client: Ibrahim Suspicious network behavior detected.	08:36:03 [HEARTBEAT] Heartbeat received from Zham Last Heartbeat: 2026-08-30 08:36:03 08:36:03 [HEARTBEAT] Heartbeat received from Zham 08:36:07 [HEARTBEAT] Heartbeat received from Zham Last Heartbeat: 2026-08-30 08:36:07 08:36:07 [HEARTBEAT] Heartbeat received from Zham 08:36:12 [HEARTBEAT] Heartbeat received from Zham 08:36:12 [HEARTBEAT] Heartbeat received from Zham Last Heartbeat: 2026-08-30 08:36:12	SERVER CONTROLS START SERVER STOP SERVER CLEAR LOGS

Figure 18.4: live activities log, server control, and security alert

Chapter 19

Discussion

NetGuard successfully combines several important computer networking concepts into one project.

The TCP client-server architecture provides the foundation of the system. The server listens on port 5000 and accepts clients.

ClientHandler demonstrates how multiple clients can be handled through threads.

HeartbeatSender and HeartbeatManager demonstrate an application-level health monitoring mechanism.

SecurityMonitor demonstrates how connection events can be analyzed to identify suspicious behavior.

The GUI makes the monitoring system easier to use and demonstrate. Instead of depending only on console messages, users can see client information, status changes, activities, and security alerts.

LogManager adds persistence to the system by saving important events to activity.log.

One of the most important achievements of the project is the integration of these components.

The system therefore represents more than a simple socket program. It demonstrates a complete small-scale monitoring workflow:

Connect → Identify → Monitor → Detect → Alert → Log

The final testing confirmed that these components work together successfully.

Chapter 20

Conclusion

NetGuard is a Java-based intelligent network security monitoring system developed as a practical Computer Networking project.

The project successfully implements TCP socket communication, client-server architecture, multiple client handling, heartbeat monitoring, client status management, security behavior detection, GUI monitoring, security alerts, and activity logging.

The server successfully starts on TCP port 5000 and accepts clients. Clients send their names and heartbeat messages. The server stores client information including IP address, connection time, status, and last heartbeat.

The heartbeat mechanism provides a simple method for monitoring client liveness. When a client disconnects, the server detects the event and changes its status to OFFLINE.

SecurityMonitor adds a basic security layer by tracking repeated connection attempts and frequent connect/disconnect behavior. Security alerts are displayed through the GUI and saved through LogManager.

The activity log mechanism successfully creates and updates.

The final testing phase successfully passed the major test cases: server startup, client connection, heartbeat, client disconnection, security alert generation, and activity logging.

Therefore, the core objectives of the NetGuard project were successfully achieved.

Although the current system has limitations, it provides a strong academic demonstration of practical networking, monitoring, multithreading, GUI development, and basic security monitoring.

Appendix A

Important Source Code

A.1 Server Port

```
1 private static final int PORT = 5000;
```

A.2 Server Socket

```
1 ServerSocket serverSocket =  
2     new ServerSocket(PORT);
```

A.3 Client Acceptance

```
1 while (true) {  
2  
3     Socket clientSocket =  
4         serverSocket.accept();  
5  
6     ClientHandler clientHandler =  
7         new ClientHandler(  
8             clientSocket,  
9             this,  
10            gui  
11        );  
12  
13     Thread clientThread =  
14         new Thread(clientHandler);  
15  
16     clientThread.start();  
17 }
```

A.4 Heartbeat Sender

```
1 public class HeartbeatSender
2     implements Runnable {
3
4     private final DataOutputStream output;
5     private volatile boolean running = true;
6
7     public HeartbeatSender(
8         DataOutputStream output) {
9
10        this.output = output;
11    }
12
13    @Override
14    public void run() {
15
16        while (running) {
17
18            try {
19
20                output.writeUTF(
21                    "HEARTBEAT"
22                );
23
24                output.flush();
25
26                System.out.println(
27                    "Heartbeat sent."
28                );
29
30                Thread.sleep(5000);
31
32            } catch (IOException e) {
33
34                System.out.println(
35                    "Heartbeat stopped: "
36                    + e.getMessage()
37                );
38
39                running = false;
40
41            } catch (InterruptedException e) {
42
43                Thread.currentThread()
44                    .interrupt();
45
46                running = false;
47            }
48        }
49    }
```

```
50  
51     public void stop() {  
52  
53         running = false;  
54     }  
55 }
```

A.5 Heartbeat Timeout

```
private static final int TIMEOUT_SECONDS = 10;
```

A.6 Security Threshold

```
private static final int WARNING_LIMIT = 3;
```

A.7 Security Alert

```
1 if (attempts >= WARNING_LIMIT) {
2
3     String alertMessage =
4         "Multiple connection attempts detected. "
5         + "Client: "
6         + clientName;
7
8     LogManager.logSecurityAlert(
9         alertMessage
10    );
11
12    if (gui != null) {
13
14        gui.securityAlert(
15            alertMessage
16        );
17    }
18 }
```

A.8 Offline Handling

```
1 clientInfo.setConnectionStatus(
2     "OFFLINE"
3 );
4
5 if (gui != null) {
6
7     gui.clientOffline(
8         clientInfo.getClientName()
9     );
10 }
```

Appendix B

LogManager Source Code

```
1 package com.netguard.server;
2
3 import java.io.File;
4 import java.io.FileWriter;
5 import java.io.IOException;
6 import java.io.PrintWriter;
7 import java.time.LocalDateTime;
8 import java.time.format.DateTimeFormatter;
9
10 public class LogManager {
11
12     private static final String LOG_FOLDER =
13         "resources/logs";
14
15     private static final String LOG_FILE =
16         "resources/logs/activity.log";
17
18     private static final DateTimeFormatter FORMATTER =
19         DateTimeFormatter.ofPattern(
20             "yyyy-MM-dd HH:mm:ss"
21         );
22
23     public LogManager() {
24
25         createLogFolder();
26
27         System.out.println(
28             "LogManager initialized. Log path: "
29             + new File(LOG_FILE)
30             .getAbsolutePath()
31         );
32     }
33
34     private void createLogFolder() {
35
36         File folder =
```

```
37         new File(LOG_FOLDER);
38
39         if (!folder.exists()) {
40
41             folder.mkdirs();
42         }
43     }
44
45     public synchronized void log(
46         String message) {
47
48         System.out.println(
49             "LOGGING: "
50                 + message
51         );
52
53         try {
54
55             File folder =
56                 new File(LOG_FOLDER);
57
58             if (!folder.exists()) {
59
60                 folder.mkdirs();
61             }
62
63             FileWriter fileWriter =
64                 new FileWriter(
65                     LOG_FILE,
66                     true
67                 );
68
69             PrintWriter writer =
70                 new PrintWriter(
71                     fileWriter
72                 );
73
74             String time =
75                 LocalDateTime.now()
76                     .format(
77                         FORMATTER
78                     );
79
80             writer.println(
81                 "[" + time + "] "
82                     + message
83             );
84
85             writer.flush();
86
87             writer.close();
```

```
88         System.out.println(  
89             "Log saved successfully."  
90         );  
91     };  
92  
93     } catch (IOException e) {  
94  
95         System.out.println(  
96             "Log Error: "  
97             + e.getMessage()  
98         );  
99     }  
100 }  
101  
102 public void logConnection(  
103     String clientName,  
104     String ipAddress) {  
105  
106     log(  
107         "CLIENT CONNECTED | Name: "  
108         + clientName  
109         + " | IP: "  
110         + ipAddress  
111     );  
112 }  
113  
114 public void logDisconnection(  
115     String clientName) {  
116  
117     log(  
118         "CLIENT DISCONNECTED | Name: "  
119         + clientName  
120     );  
121 }  
122  
123 public void logWarning(  
124     String message) {  
125  
126     log(  
127         "SECURITY WARNING | "  
128         + message  
129     );  
130 }  
131  
132 public void logSecurityAlert(  
133     String message) {  
134  
135     log(  
136         "SECURITY ALERT | "  
137         + message  
138     );
```

```
139     }  
140 }
```

Appendix C

Final Test Evidence

C.1 Server Startup Evidence

```
=====
NetGuard Server
=====
Server started successfully.
Port: 5000
Waiting for clients...
```

C.2 Client Connection Evidence

```
New client connection received.

Security Monitor: Checking client Ibrahim
Connection Attempts: 1
Client verified: Ibrahim

===== Connected Clients =====

1. Ibrahim
IP: 127.0.0.1
Status: ONLINE
```

C.3 Heartbeat Evidence

Heartbeat received from Ibrahim

Client: Ibrahim
IP: 127.0.0.1
Status: ONLINE
Last Heartbeat:
2026-08-27 14:53:57

C.4 Offline Evidence

Client disconnected: Ibrahim
Status changed to OFFLINE

Security Monitor:
Client disconnected

Client: Ibrahim
Disconnect Count: 1

Client Offline: Ibrahim

C.5 Security Alert Evidence

Connection Attempts: 3

SECURITY ALERT

Suspicious connection behavior detected!

Client: Ibrahim

Multiple connection attempts detected.

Disconnect Count: 3

SECURITY ALERT

Frequent connect/disconnect behavior detected!

Client: Ibrahim

C.6 Logging Evidence

```
LogManager initialized. Log path:  
...\resources\logs\activity.log
```

```
LOGGING:  
SECURITY ALERT |  
Frequent connect/disconnect behavior detected.  
Client: Ibrahim
```

```
Log saved successfully.
```

Appendix D

Final Demonstration Checklist

No. Demonstration Item Status

- 1 ServerMain starts successfully PASS
- 2 Server GUI opens PASS
- 3 Server starts on port 5000 PASS
- 4 Client connects to server PASS
- 5 Client name is received PASS
- 6 Client IP is detected PASS
- 7 Client status becomes ONLINE PASS
- 8 Heartbeat is received PASS
- 9 Last heartbeat time is updated PASS
- 10 Client disconnect is detected PASS
- 11 Client becomes OFFLINE PASS
- 12 Connection attempts are counted PASS
- 13 Disconnect count is maintained PASS

- 14 Security alert is generated PASS
 - 15 Security alert appears in GUI PASS
 - 16 Live activity is displayed PASS
 - 17 LogManager initializes PASS
 - 18 activity.log is created PASS
 - 19 Security alert is saved to log PASS
 - 20 Final testing completed PASS
-

Bibliography

- William Stallings, *Network Security Essentials: Applications and Standards*, 6th Edition, Pearson, 2017.
- James F. Kurose and Keith W. Ross, *Computer Networking: A Top-Down Approach*, 8th Edition, Pearson, 2021.
- Behrouz A. Forouzan, *Data Communications and Networking*, 5th Edition, McGraw-Hill Education, 2017.
- Oracle Corporation, *Java Platform, Standard Edition Documentation*, 2026.
- Oracle Corporation, *Java Networking and Socket Documentation*, Java SE 21 Documentation, 2026.
- Oracle Corporation, *Java Thread and Concurrency Documentation*, Java SE 21 Documentation, 2026.
- Oracle Corporation, *Creating a GUI With Swing*, Oracle Java Tutorials, 2026.
- Jon Postel, *Transmission Control Protocol*, RFC 793, 1981.
- Robert Braden, *Requirements for Internet Hosts – Communication Layers*, RFC 1122, 1989.
- Andrew S. Tanenbaum and David J. Wetherall, *Computer Networks*, 5th Edition, Pearson, 2011.
- Bruce Eckel, *Thinking in Java*, 4th Edition, Prentice Hall, 2006.

Appendix E

CEA and CEP

Complex Engineering Problem (CEP)

CEP Attribute	Problem Details / Justification	Status
P1 – Depth of Knowledge	The development of NetGuard requires knowledge of computer networking, TCP/IP communication, Java socket programming, multithreading, GUI development, and basic network security concepts.	✓
P2 – Conflicting Requirements	The system needs to monitor clients continuously while maintaining responsive GUI updates, heartbeat processing, security monitoring, and activity logging without affecting overall performance.	✓
P3 – Depth of Analysis	NetGuard analyzes connection attempts, client disconnections, heartbeat information, and connection status to identify unusual or suspicious network behavior.	✓
P4 – Familiarity of Issues	The project deals with client disconnection, connection failure, missing heartbeat signals, and repeated connection attempts.	✓
P5 – Applicable Codes and Standards	The system uses standard TCP socket communication and Java networking APIs. However, it does not implement a specific industry-level security compliance standard.	×
P6 – Stakeholder Involvement	The NetGuard GUI provides information to the system user about connected clients, online/offline status, heartbeat activity, security alerts, and live activities.	✓
P7 – Interdependence	Server, ClientHandler, HeartbeatManager, SecurityMonitor, ServerGUI, ClientInfo, and LogManager work together to provide complete real-time monitoring.	✓
P8 – Consequences	Incorrect heartbeat monitoring or failure to identify suspicious behavior may result in inaccurate client status or missed security events.	✓

Complex Engineering Activities (CEA)

CEA Attribute	Problem Details / Justification	Status
A1 – Range of Resources	The project uses Java JDK, IntelliJ IDEA, Java networking libraries, Swing GUI components, file handling, threads, sockets, and local system resources.	✓
A2 – Interactions	The server communicates with multiple clients while ClientHandler, HeartbeatManager, SecurityMonitor, ServerGUI, and LogManager interact during system execution.	✓
A3 – Creative and Innovative	NetGuard combines socket communication, heartbeat monitoring, connection-attempt analysis, automatic security alerts, GUI visualization, and activity logging.	✓
A4 – Leadership and Management	The project was organized into separate modules. However, formal project-management methodologies or dedicated management tools were not used.	×
A5 – Social, Environmental and Economic Impact	The project provides a lightweight software-based monitoring solution without specialized networking hardware. However, formal large-scale impact assessment was not performed.	×
A6 – Professionalism	The implementation follows modular Java programming practices, exception handling, synchronization, multithreading, logging, and systematic testing.	✓
A7 – Communication	The graphical interface communicates client information, online/offline status, heartbeat events, security alerts, and live activity logs.	✓
A8 – Lifelong Learning	The development required learning Java socket programming, multithreading, Swing GUI development, heartbeat monitoring, security detection, and file-based logging.	✓